

NoSQL Data Modeling Patterns

Chapter 06 - Sub-Chapter 08

NoSQL Databases Series

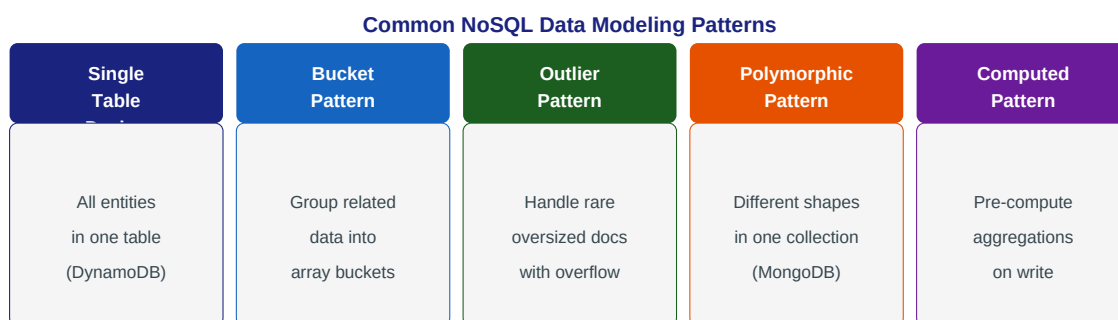
Introduction	Think in queries, 3-step workflow
Embed vs Reference	The core document design decision
Single Table Design	DynamoDB: all entities in one table
Computed & Bucket	Pre-aggregate on write, bucket time-series

1 NoSQL Data Modeling

Think in queries, not in entities

The biggest mistake developers make when switching from SQL to NoSQL is trying to use the same modeling approach. In SQL, you normalize data into entities and let queries handle the relationships. In NoSQL, you start with the queries and design the data to serve them.

This chapter covers the fundamental patterns that apply across MongoDB, DynamoDB, and Cassandra — the core decisions that separate fast, scalable NoSQL from a slow, expensive mess.



The NoSQL modeling workflow (3 steps):

Step 1 — **List all access patterns** before writing any schema. 'Get user by ID', 'Get all orders for user, newest first', 'Get product by SKU'. Every query you need.

Step 2 — **Choose how to group data** to serve each access pattern in a single read. Avoid multi-document reads wherever possible.

Step 3 — **Accept controlled duplication**. In NoSQL, storing the same data in two places is not a bug — it is a feature if it means queries are faster.

2 Embed vs Reference

The most important design decision in document databases

Every relationship in a document database has the same choice: embed the related data inside the parent document, or store a reference (like a foreign key) to a separate document.

Embed vs Reference — The Most Important NoSQL Design Decision

EMBED (denormalize) — One document

```
{ "orderId": "ord-1",
  "userId": "alice",
  "items": [           <-- embedded!
    { "sku": "BOOT", "qty": 1, "price": 89.99 },
    { "sku": "SOCK", "qty": 2, "price": 7.50 }
  ],
  "address": {         <-- embedded!
    "street": "123 Main", "city": "NY"
  }
}
```

✓ 1 read, atomic write, cache-friendly

REFERENCE — Two documents + join

```
{ "orderId": "ord-1",
  "userId": "user-ref-123", <-- reference
  "itemIds": ["item-a", "item-b"] <-- refs
}

{ "userId": "user-ref-123",
  "name": "Alice",
  "email": "alice@example.com"
}

// Need 2 queries or $lookup
```

✗ 2+ reads, eventual consistency risk

When to EMBED:

- ✓ Data is always read together (order + its items are always displayed together).
- ✓ Data has a 'contains' relationship (address is part of user — not standalone).
- ✓ The embedded data does not grow without bound (max 50 items in an order).
- ✓ Embedded data is not accessed independently (you never query items without their order).

When to REFERENCE:

- ✓ The related data is frequently updated independently (user profile updated separately from orders).
- ✓ The embedded array could grow without bound (a user's entire purchase history — could be millions).
- ✓ The data is shared across many documents (a product referenced by millions of orders — update once).
- ✓ You need to query the related data independently (find all reviews for a product, not through orders).

The Outlier Pattern — handling the unbounded array:

```
// Problem: a celebrity Twitter account has 50 million followers
// Embedding all follower IDs in one document would be gigabytes!

// Outlier Pattern: embed up to a limit, overflow to separate documents
{
  "_id": "user-elon",
  "name": "Elon Musk",
  "follower_count": 50000000,
  "followers": ["user-1", "user-2", ... "user-1000"], // first 1000
  "has_more_followers": true // flag indicating overflow
}

// Overflow documents (referenced by user ID):
{
```

```
"_id": "followers-user-elon-page-2",  
"user_id": "user-elon",  
"page": 2,  
"followers": ["user-1001", ... "user-2000"]  
}  
// Most users fit in one document; only outliers (celebrities) use overflow
```

3 Single Table Design (DynamoDB)

All entity types in one table

DynamoDB charges per read request, not per join. Since DynamoDB has no joins at all, the way to fetch related data in one request is to put it in the same partition. Single Table Design (STD) is the pattern that makes this possible.

DynamoDB Single Table Design — All Entities, One Table

PK	SK	Type	data...
USER#alice	PROFILE#alice	User	name=Alice, email=...
USER#alice	ORDER#2024-01-15#001	Order	status=CONFIRMED, \$120
USER#alice	ORDER#2024-01-16#002	Order	status=SHIPPED, \$89
PRODUCT#BOOT-42	METADATA	Product	name=Boot, price=\$89.99
ORDER#2024-01-15#001	ITEM#BOOT-42	OrderItem	qty=1, price=\$89.99

All entity types co-located — enables 1 query for all related data

The key insight of Single Table Design:

Use generic PK and SK names (PK, SK) and give them type-prefixed values like 'USER#alice' and 'ORDER#2024-01-15'. This lets you store any entity type in the same table and query related items by partition in a single request.

```
import software.amazon.awssdk.services.dynamodb.model.*;
import java.util.Map;

// Fetch user profile AND all their orders in ONE query
// Because all alice's data shares PK = "USER#alice"
QueryRequest request = QueryRequest.builder()
    .tableName("ApplicationTable")
    .keyConditionExpression("PK = :pk")
    .expressionAttributeValues(Map.of(
        ":pk", AttributeValue.fromS("USER#alice")
    ))
    .build();
// Returns: user profile (SK=PROFILE#alice) + all orders (SK=ORDER#...)
// One network round trip -- no joins needed

// OR: fetch just the orders (range on SK)
QueryRequest ordersOnly = QueryRequest.builder()
    .tableName("ApplicationTable")
    .keyConditionExpression("PK = :pk AND begins_with(SK, :prefix)")
    .expressionAttributeValues(Map.of(
        ":pk", AttributeValue.fromS("USER#alice"),
        ":prefix", AttributeValue.fromS("ORDER#")
    ))
    .build();
```

4 Computed and Bucket Patterns

Pre-aggregate on write, bucket time-series

Computed Pattern — pre-calculate on write, not on read:

If you display a product rating on every page view but recalculate it by averaging all reviews on every request, you are doing expensive work millions of times. The Computed Pattern stores the aggregate and updates it each time a review is added.

```
// MongoDB: Computed Pattern -- pre-aggregate product rating

// Product document holds pre-computed stats
{
  "_id": "product-boot-42",
  "name": "Trail Boot",
  "review_count": 1247,
  "rating_sum": 5423.5,
  "avg_rating": 4.35      // pre-computed -- read is instant!
}

// When a new review is added, atomically update the aggregate:
db.products.updateOne(
  { _id: "product-boot-42" },
  {
    $inc: { review_count: 1, rating_sum: 4.5 },
    $set: { avg_rating: (existingSum + 4.5) / (existingCount + 1) }
  }
);
// One write updates the aggregate -- no re-computation on reads
```

Bucket Pattern — grouping time-series data in MongoDB:

Storing one MongoDB document per IoT measurement creates millions of tiny documents — inefficient indexing, high storage overhead. The Bucket Pattern groups measurements into time-bucket documents.

```
// BAD: one document per reading (millions of documents)
{ "_id": ObjectId(), "sensorId": "s-42", "ts": ISODate("2024-01-15T09:00:00Z"), "value": 23.1 }
{ "_id": ObjectId(), "sensorId": "s-42", "ts": ISODate("2024-01-15T09:01:00Z"), "value": 23.4 }
// ... 1 million more documents per day per sensor

// GOOD: Bucket Pattern -- one document per sensor per hour
{
  "_id": "s-42_2024-01-15-09",      // sensor + hour
  "sensorId": "s-42",
  "hour": ISODate("2024-01-15T09:00:00Z"),
  "count": 60,
  "min": 22.9, "max": 23.7, "sum": 1400.4,
  "readings": [
    { "ts": ISODate("2024-01-15T09:00:00Z"), "v": 23.1 },
    { "ts": ISODate("2024-01-15T09:01:00Z"), "v": 23.4 }
    // ... 58 more
  ]
}
// 60x fewer documents -- indexes are smaller, queries faster
// Adding a reading: $push to the readings array, $inc count/sum
```

Polymorphic Pattern — different shapes in one collection:

```
// MongoDB: store different product types in one 'products' collection
// Each document has a 'type' field and type-specific attributes

// Electronics product
{ "_id": "...", "type": "electronics", "name": "Laptop",
  "cpu": "i7", "ram_gb": 16, "storage_gb": 512 }

// Clothing product
{ "_id": "...", "type": "clothing", "name": "Trail Boot",
  "sizes": ["S", "M", "L"], "material": "leather", "waterproof": true }

// Food product
{ "_id": "...", "type": "food", "name": "Organic Coffee",
  "weight_kg": 0.5, "expiry_days": 365, "allergens": ["none"] }

// Query all products (polymorphic -- works for all types):
db.products.find({ "category": "outdoors" })

// Query type-specific attribute:
db.products.find({ "type": "clothing", "waterproof": true })
```

Key Rules

- Start with access patterns: list every query your app needs before designing any schema.
- Embed for 'contains' relationships with bounded size; reference for shared or unbounded data.
- DynamoDB Single Table Design: prefix keys with entity type (USER#, ORDER#) for co-location.
- Computed Pattern: maintain aggregates on write -- never recalculate avg/count on every read.
- Bucket Pattern: group time-series into hourly/daily documents to reduce index size 60x.